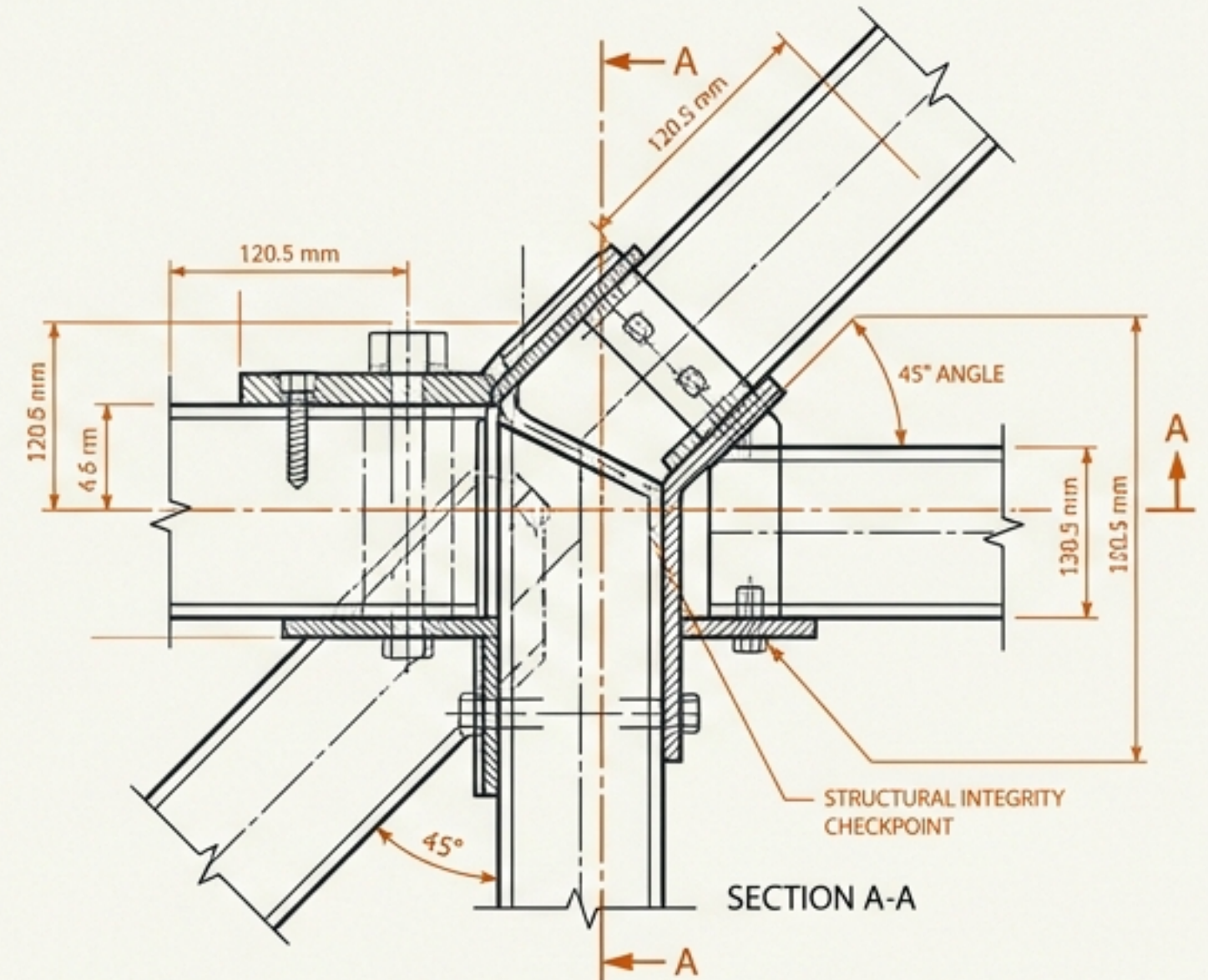
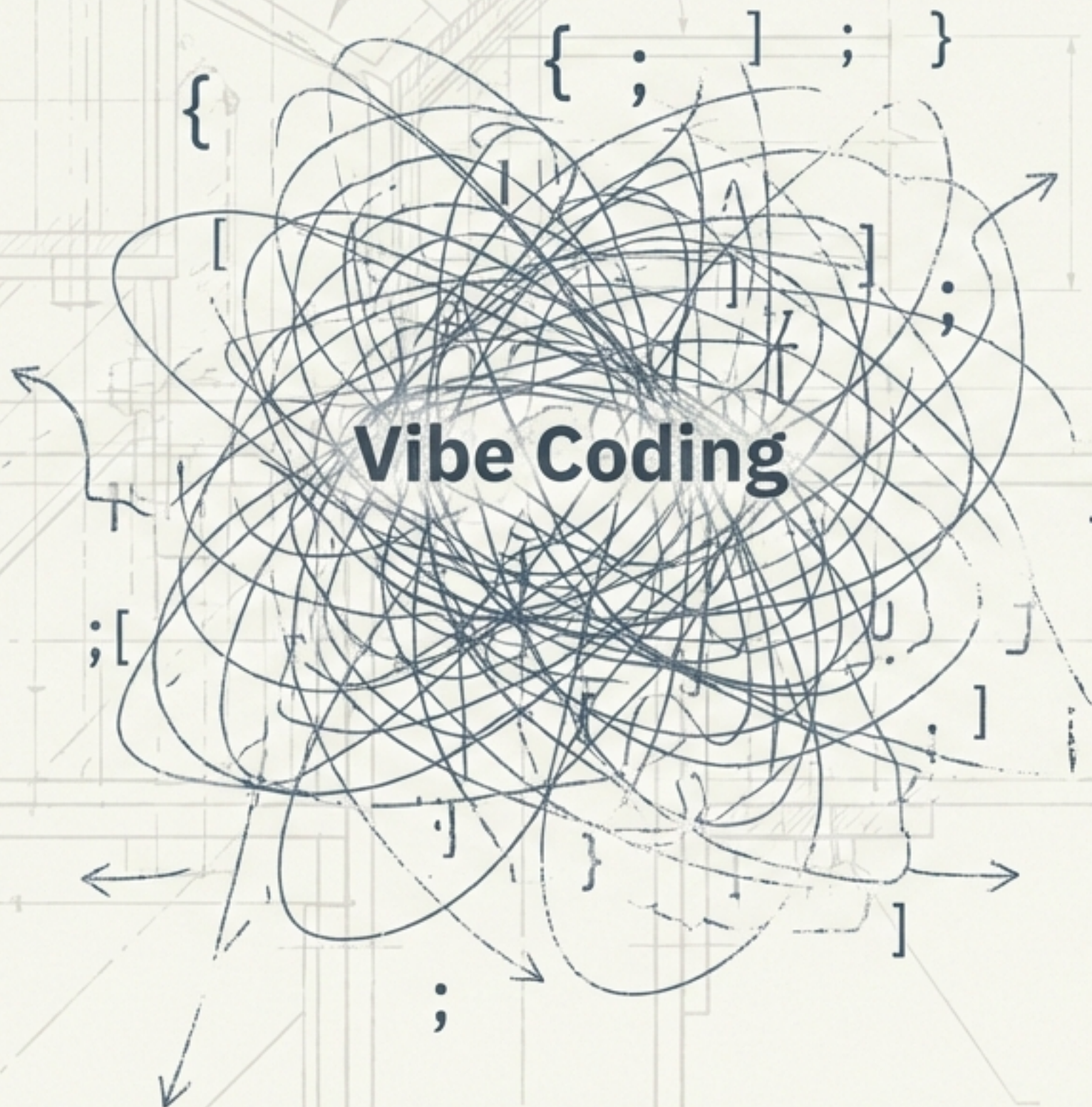


วิศวกรรมบริบทขั้นสูงสำหรับ Agent

ก้าวข้าม 'Vibe Coding' สู่การส่งมอบโค้ดระดับ Production ในระบบที่มีความซับซ้อน

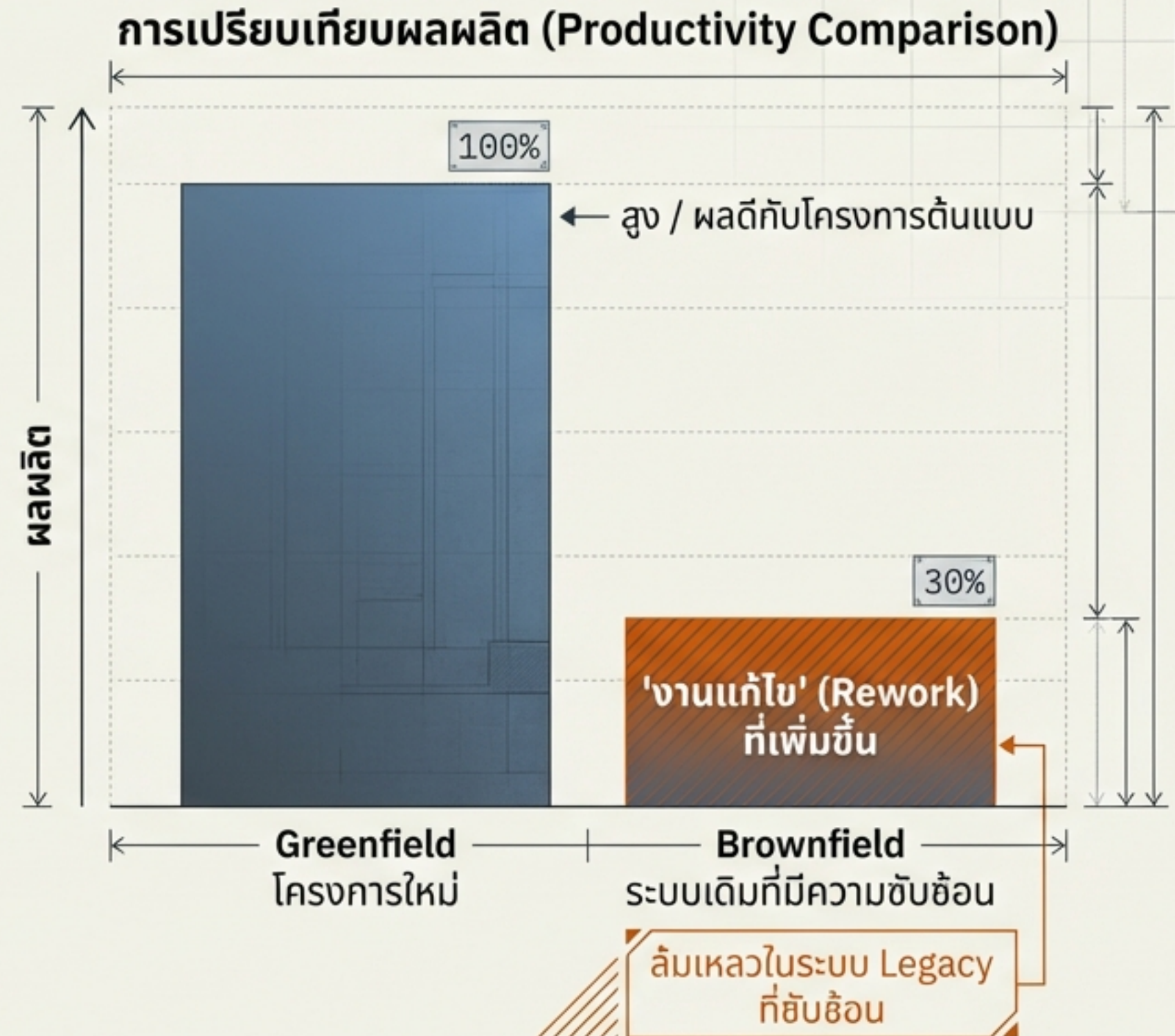


บทนำสู่ระเบียบวิธีทางวิศวกรรมที่แข็งแกร่ง

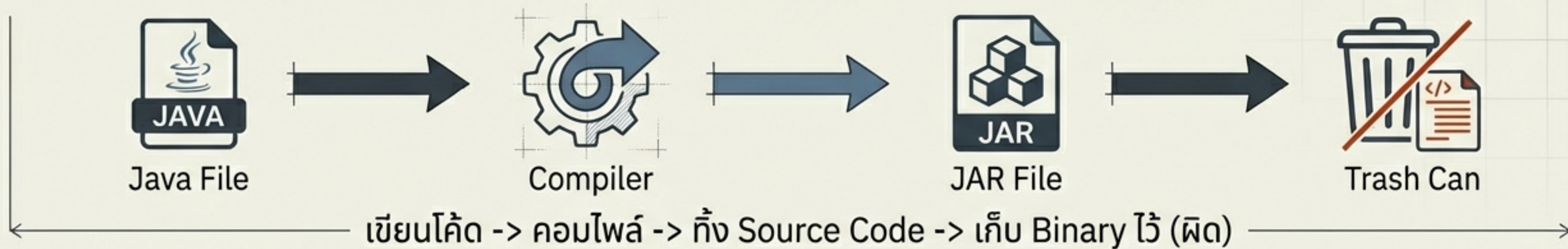
ความจริงในปัจจุบัน: วิศวกรรม AI มักให้ผลลัพธ์ที่สวนทาง

ข้อมูลจากผลการศึกษาของ Stanford (นักพัฒนา 100,000 คน):

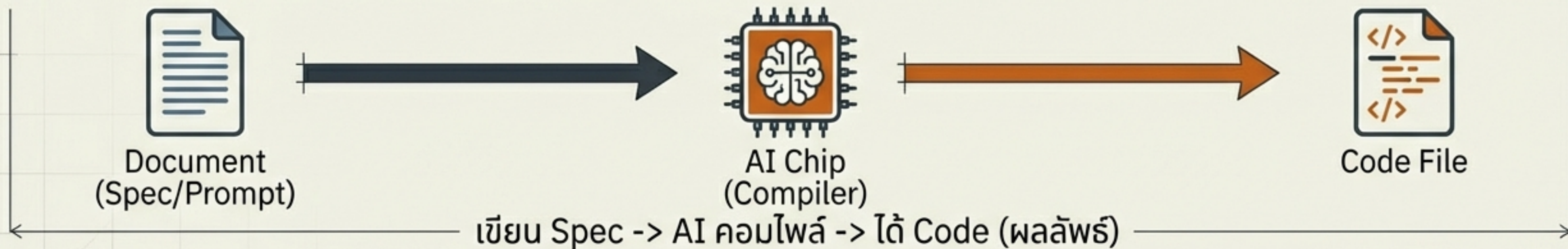
- การเขียนโค้ดด้วย AI มักนำไปสู่ 'การรีองานใหม่' (Rework) และความไม่เรียบร้อย
- **Greenfield** vs. **Brownfield**: ได้ผลดีกับโครงการต้นแบบ แต่ล้มเหลวในระบบ Legacy ที่ซับซ้อน
- ปัจจัยความ 'หละหลวม' (Slop): ในงานที่ซับซ้อน การใช้ AI อย่างไม่มีแบบแผนทำให้วิศวกรอาวุโสทำงานช้าลง



เปลี่ยนกรอบความคิด: ข้อกำหนด (Spec) คือ Source Code ที่แท้จริง

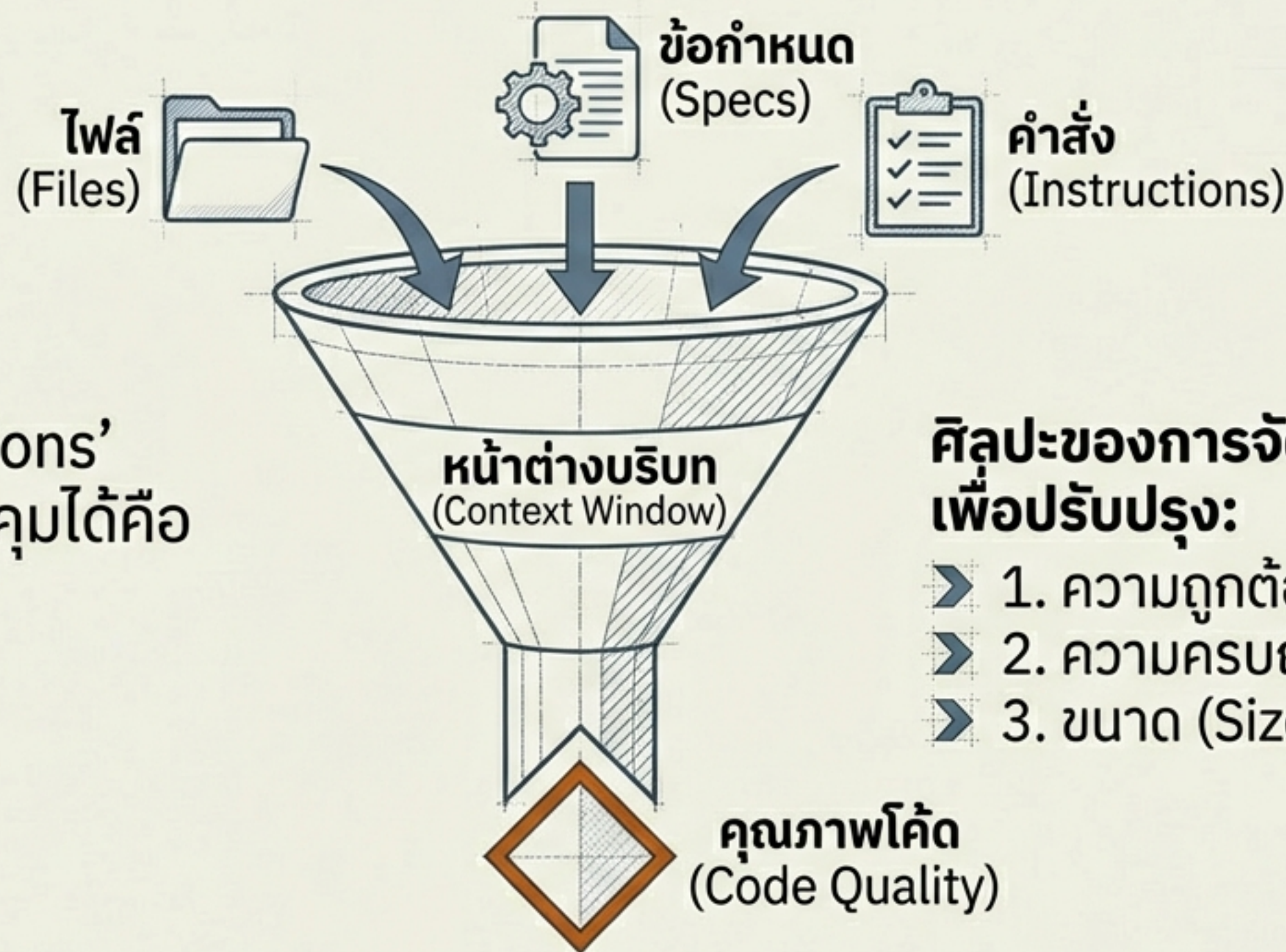


เวิร์กโฟลว์ที่ถูกต้อง - Context Engineering



“ในอนาคตที่ AI เป็นผู้เขียนโค้ด คำอธิบายสิ่งที่เราต้องการ (Spec) คือสินทรัพย์ที่สำคัญที่สุด ไม่ใช่ตัวโค้ดเอง”

นิยามของวิศวกรรมบริบท (Context Engineering)



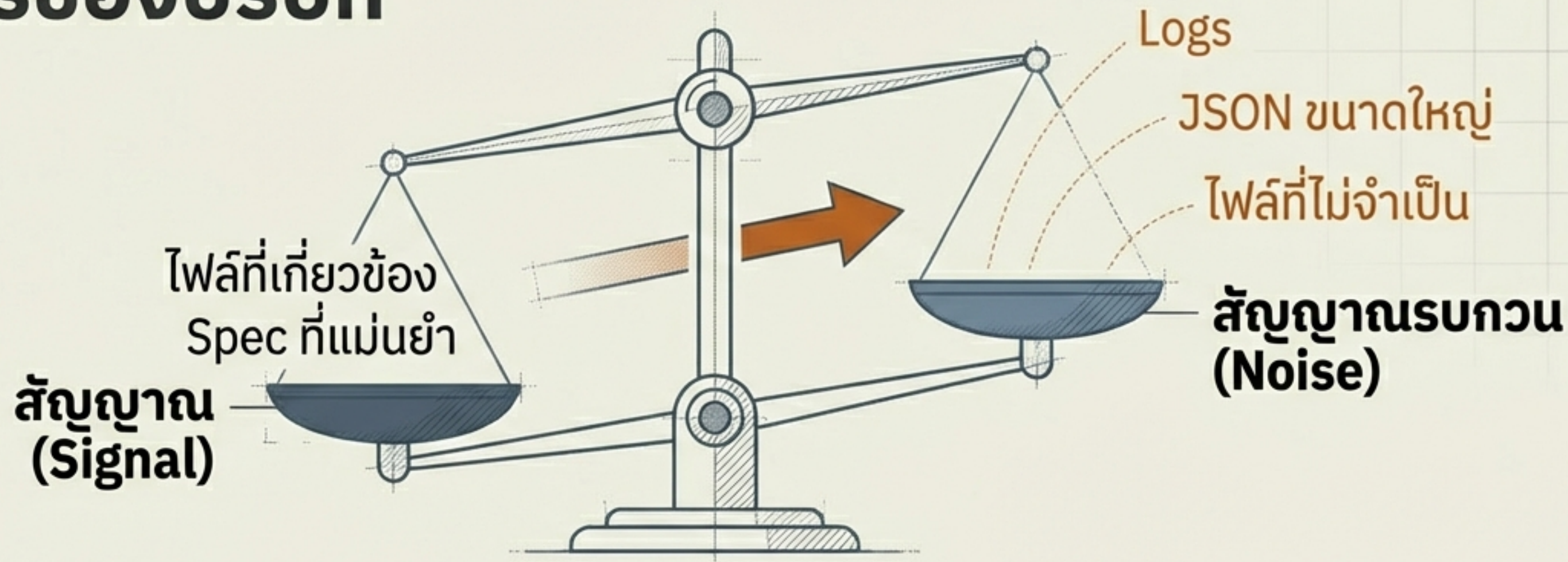
LLM คือ 'Pure Functions'
ตัวแปรเดียวที่คุณควบคุมได้คือ
Input (บริบท)

ศิลปะของการจัดการหน้าต่างบริบท
เพื่อปรับปรุง:

- 1. ความถูกต้อง (Correctness)
- 2. ความครบถ้วน (Completeness)
- 3. ขนาด (Size)

ขยะเข้า = ขยะออก (Garbage In = Garbage Out)

สมการของบริบท



เป้าหมาย: ใช้ Token น้อยที่สุดเพื่อให้ได้ความหนาแน่นของสัญญาณ (Signal Density) สูงสุด

ลำดับขั้นของบริบทที่แย่:

1. ข้อมูลผิด (Bad Info) *อันตรายที่สุด นำไปสู่ **Hallucination***
2. ข้อมูลขาดหาย (Missing Info)
3. สัญญาณรบกวน (Noise) *ทำให้ความสนใจเจือจาง*

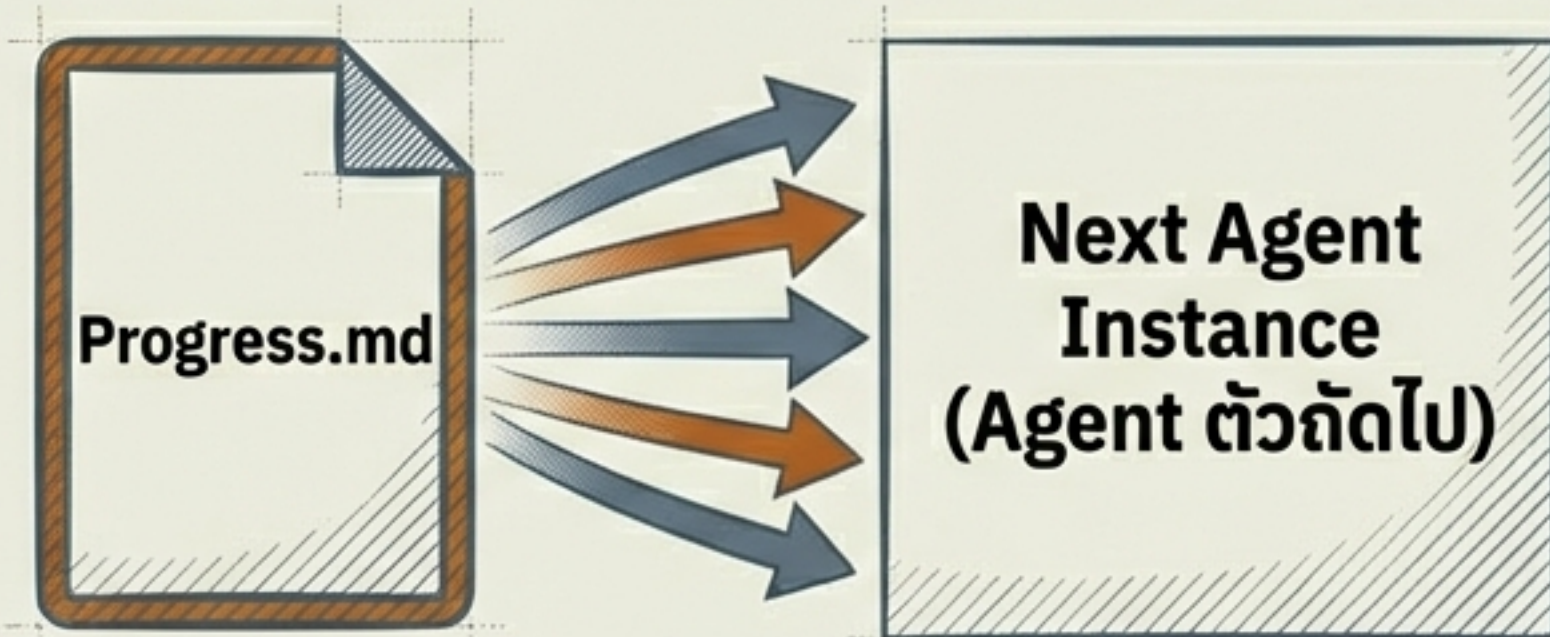
เลิก ‘ตะโกนใส่ AI’

วิธีแบบเดิม (Naive Looping)

การวนลูปสั่งงานแบบ “ไม่ใช่ ทำแบบนี้”,
“หยุด นายทำผิด” จะทำให้
Context เต็มไปด้วยข้อผิดพลาด

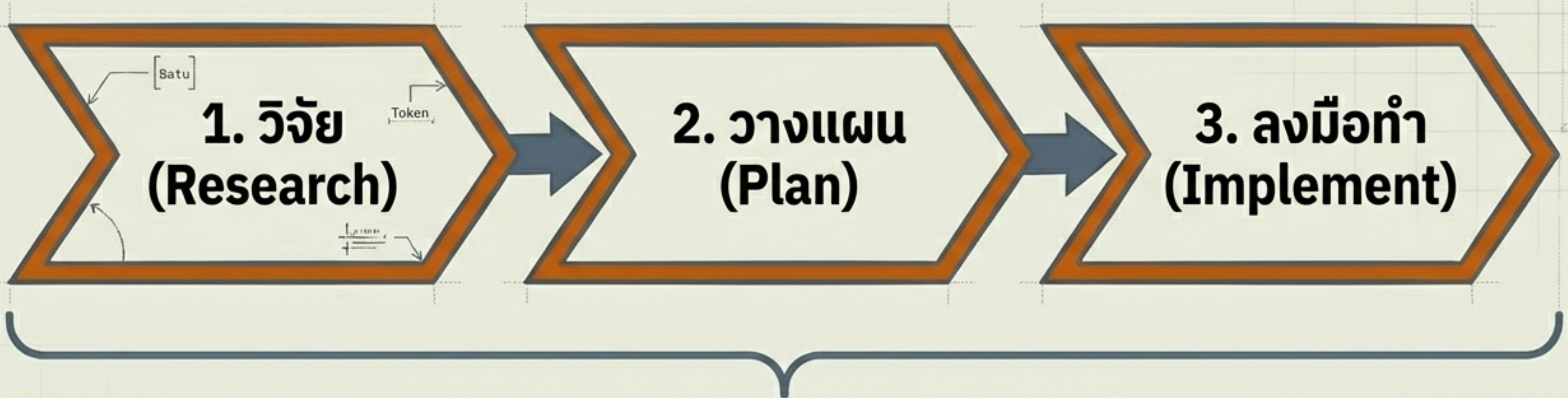
กลยุทธ์: การบีบอัดอย่างตั้งใจ (Intentional Compaction)

อย่าใช้คำสั่ง ‘/compact’ ทั่วไป
แต่ให้บังคับ Agent เขียนไฟล์สรุปสถานะ



ไฟล์นี้จะสร้างความจำที่ผิดพลาดและส่งต่อสถานะ
ที่ถูกต้องไปยัง Agent ตัวใหม่

กระบวนการทำงานแบบเส้นตรง: วิจัย -> วางแผน -> ลงมือทำ



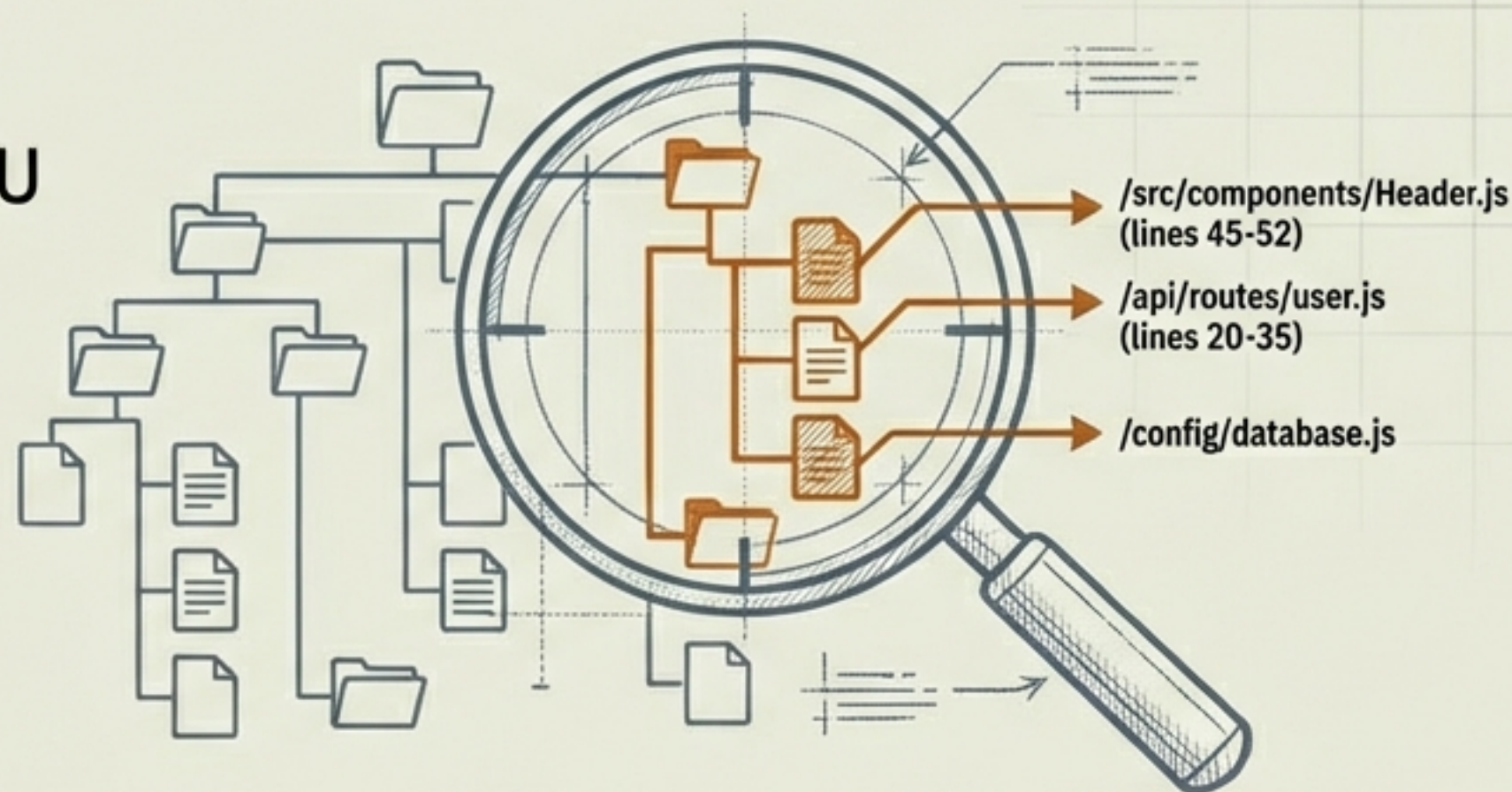
กฎเหล็ก: การใช้ Context < 40% ต่อขั้นตอน

- ห้ามข้ามขั้นตอนจนกว่าผลลัพธ์ของขั้นตอนก่อนหน้าจะได้รับการตรวจสอบ
- ล้าง Context (Clear Context) ระหว่างขั้นตอน
- สร้างกระบวนการที่แน่นอน (Deterministic) สำหรับโมเดลที่ไม่แน่นอน

ระยะที่ 1: การวิจัย (ทำความเข้าใจพื้นที่)

เป้าหมาย: เข้าใจการทำงานของระบบและการไหลของข้อมูล *ก่อน* ที่จะเริ่มแกะต้องโค้ด

Input (Prompt):



เป้าหมาย: เข้าใจการทำงานของระบบและการไหลของข้อมูล *ก่อน** ที่จะเริ่มแกะต้องโค้ด

Output (ผลลัพธ์): "สั่งให้ Agent ค้นหาไฟล์ที่เกี่ยวข้องและหมายเลขบรรทัด"

Output (ผลลัพธ์): "รายการของไฟล์และบรรทัดที่เจาะจง"

Agent ฝ่ายปฏิบัติการ **ไม่ต้องค้นหาจาก 100 ไฟล์** แต่ควรได้รับการบอกตำแหน่งที่ชัดเจน

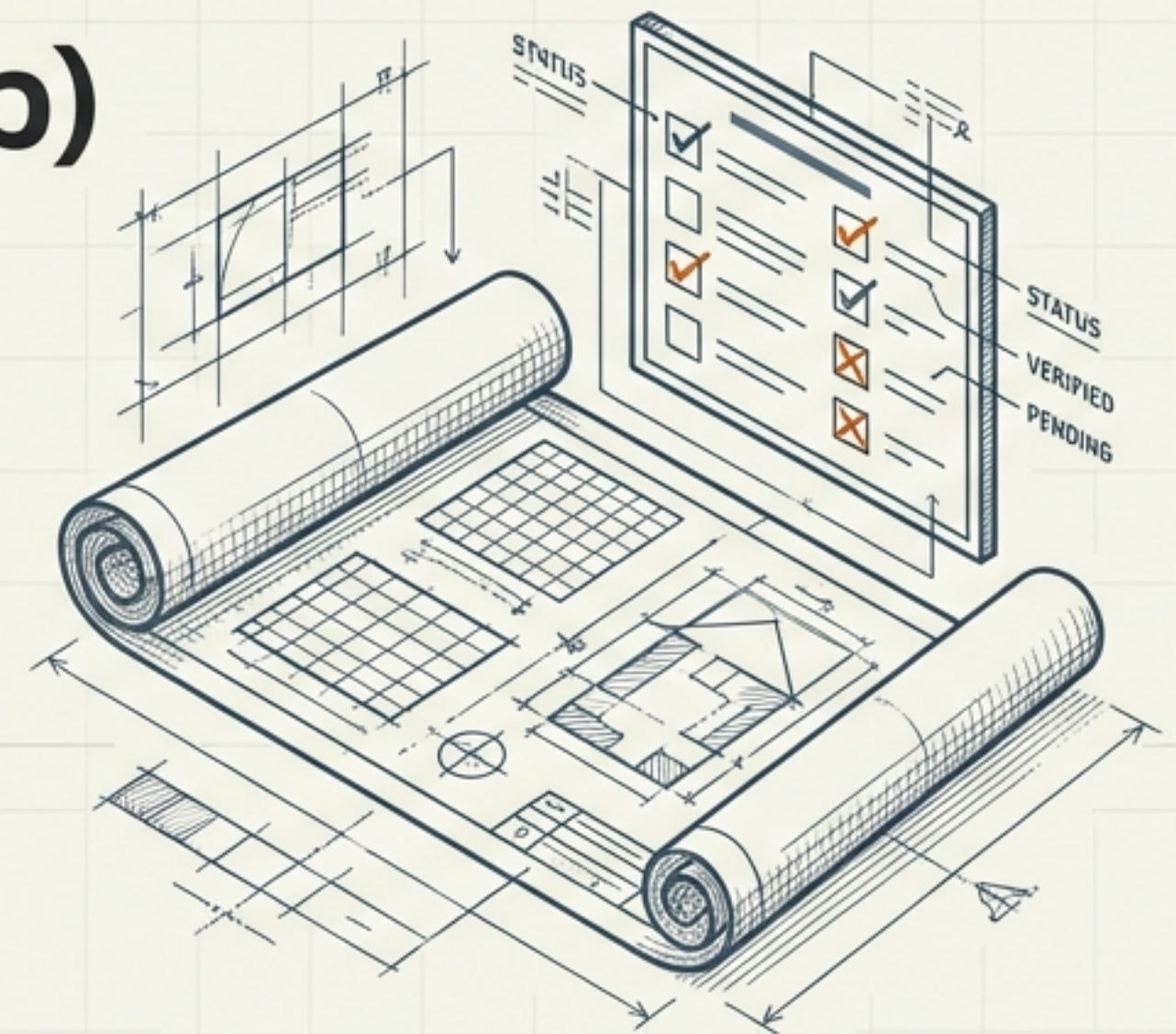
ระยะที่ 2: การวางแผน (พิมพ์เขียว)

เป้าหมาย: ระบุงูกการเปลี่ยนแปลง การทดสอบ และขั้นตอนการตรวจสอบอย่างชัดเจน

กฎ: 'ถ้าแผนงานดี ผมไม่เคยต้องตะโกนใส่ AI'

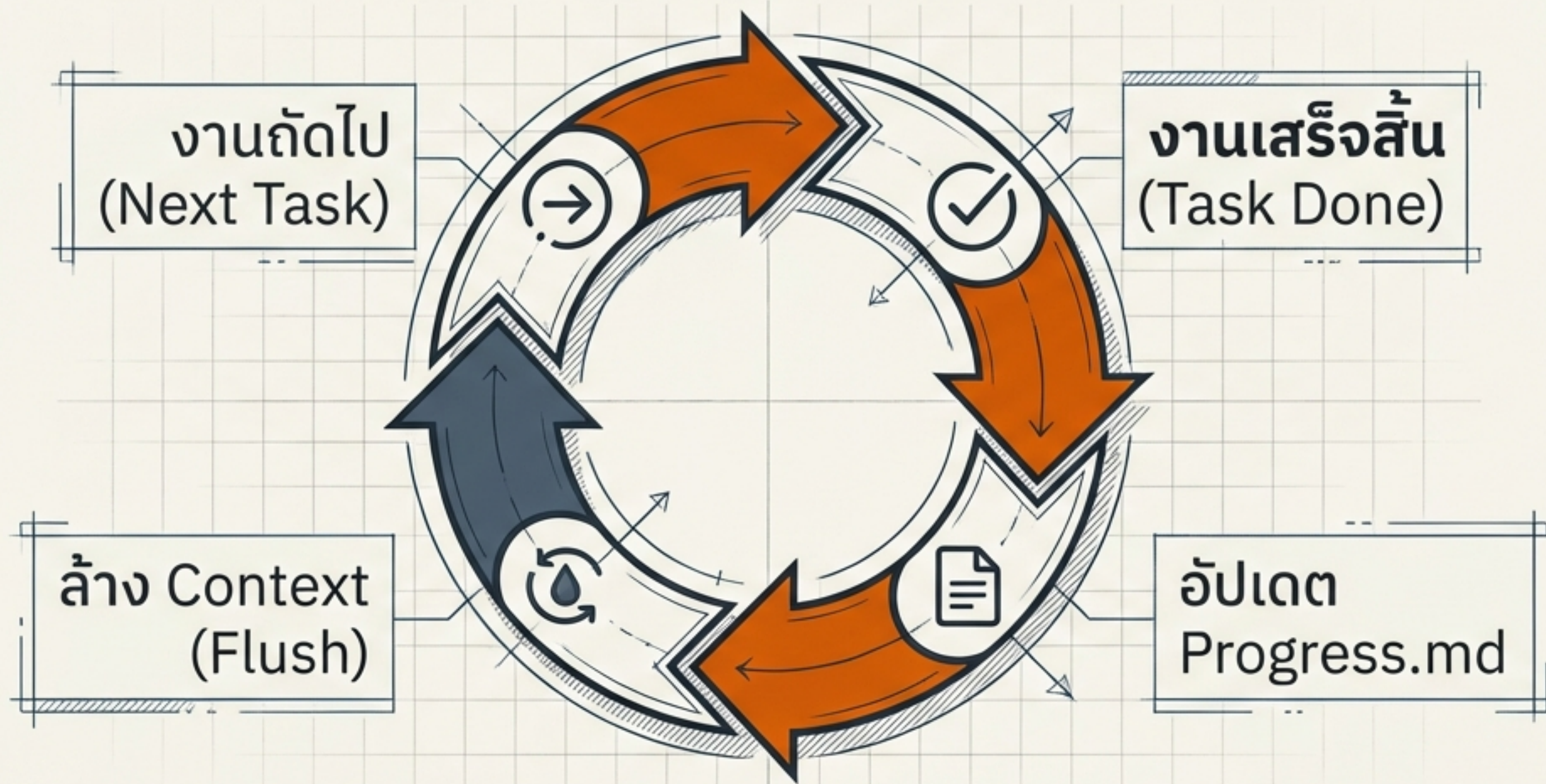
องค์ประกอบของแผนงาน:

- ไฟล์ที่ต้องแก้ไข
- Snippet ของการเปลี่ยนแปลง
- กลยุทธ์การตรวจสอบ (Verification):
เราจะรู้ได้อย่างไรว่ามันทำงานได้?



แผนงาน (The Plan)
คือความจริงเพียงหนึ่งเดียว
(Source of Truth)

ระยะที่ 3: การลงมือทำ (การก่อสร้าง)



เทคนิค: **Constant Context Flushing**

- **เหตุผล:** อย่าปล่อยให้หน้าตาของบริบทปนเปื้อนด้วย Error เก่า หรือเมื่องานเสร็จสิ้นแล้ว
- **ผลลัพธ์:** Agent จะทำงานด้วยความสนใจสูงสุด (**Peak Attention**) ในงานปัจจุบันเสมอ

ลำดับชั้นของข้อผิดพลาด (The Hierarchy of Errors)



จงใช้เวลาของคุณไปกับการตรวจสอบ **‘การวิจัย’** และ **‘แผนงาน’** ไม่ใช่การดูไวยากรณ์โค้ด

“นิยามใหม่ของการ Review Code”



ปัญหา: คุณไม่สามารถตรวจสอบ PR โค้ด Go จำนวน 2,000 บรรทัดที่เขียนโดย AI ได้อย่างมีประสิทธิภาพ



ทางออก: ตรวจสอบที่ ‘แผนงาน’ (Spec) แทน

แนวคิด: Mental Alignment (ความเข้าใจตรงกันทางความคิด)
หาก Spec ถูกต้องและ Test ผ่าน จงเชื่อใจใน ‘Compiled Binary’ (ตัวโค้ด)

“ผมอ่าน Test ทั้งหมด แต่ผมไม่อ่านโค้ดทุกบรรทัดอีกต่อไป”

กรณีศึกษา: โค้ด Rust 300,000 บรรทัด (BAML)

One-Shot Fix บน Codebase ขนาดใหญ่

ความท้าทาย: แก้ไขระบบ Legacy (Brownfield) ที่ซับซ้อน

ผลลัพธ์: PR ถูก Merge โดย CTO ก่อนที่จะรู้ว่าเป็นไอเดียของ AI

ตัวอย่างเสริม: งาน 35,000 บรรทัดใน 1 วัน
(เทียบเท่าผลงานมนุษย์ 2 สัปดาห์)

JetBrains Mono code {
 codorys
 setre: folla.nawzeckes,
 fixes: code, aigneede, +619);
};

APPROVED

1 Day (AI) vs 2 Weeks (Human)

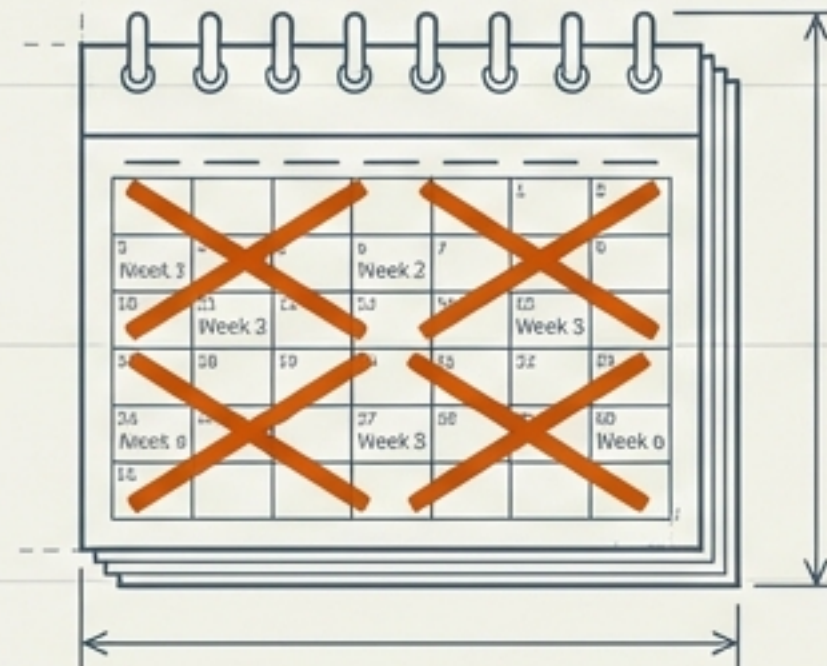
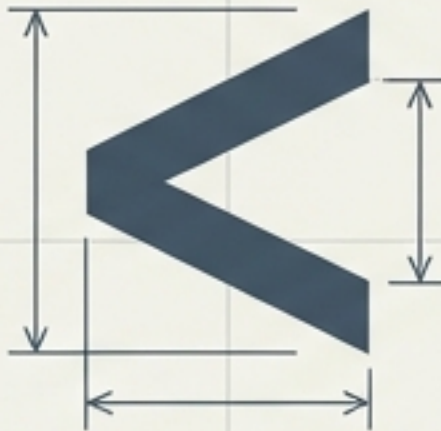
สิ่งนี้ได้ผลจริงในโปรเจกต์ Brownfield หากทำ Context Engineering อย่างแข็งแกร่ง

เศรษฐศาสตร์ของวิศวกรรมบริษัท

ค่าใช้จ่าย API สูง
(Token Usage)



เวลาทำงานของวิศวกรหลายสัปดาห์
(Engineering Weeks)



การใช้ Token ปริมาณมาก (ต้นทุนสูง) เป็นเรื่องที่ยอมรับได้
เพราะมันทดแทนค่าแรงวิศวกรได้มหาศาล

“ผมไม่ได้เปิดไฟล์อื่นนอกจาก Markdown เลยมาเป็นเวลา 2 เดือนแล้ว”

อนาคตของการพัฒนาซอฟต์แวร์

คำทำนาย:

Coding Agents
จะกลายเป็นสินค้าโภคภัณฑ์
(Commodity)
โมเดลทุกตัวจะฉลาด
เหมือนกันหมด

ความได้เปรียบในการแข่งขัน
(The Moat):

ความสามารถของทีมใน
การสื่อสารผ่าน 'Spec'
และรักษาวินัยใน
'Workflow'

เริ่มใช้ Spec-First Development วันนี้:
เปลี่ยนจุดโฟกัสจากการเขียนโค้ด ไปสู่วิศวกรรมบริบท